

COMO CREAR UN MONOREPO CON GIT

Unificar backend y frontend en un solo repositorio
para commitear todo desde una sola vista

Basado en la experiencia con el proyecto GESTION_TICKET_EPEM

Septiembre 2026

PASO 1: PARARSE EN LA RAIZ DEL PROYECTO

Abrí una terminal (Git Bash, VS Code o Fork) y navegá hasta la carpeta que contiene tanto el backend como el frontend.

```
cd "/c/Users/tu-usuario/ruta/PORTAL_DE_HERRAMIENTAS"
```

NOTA: Usa comillas dobles si la ruta tiene espacios.

PASO 2: ELIMINAR EL .GIT INTERNO DEL BACKEND

Si el backend ya fue inicializado como un repositorio git aparte (tiene su propia carpeta .git), hay que eliminar ese control de versiones interno. De lo contrario, el monorepo raiz no detectara los archivos del backend.

```
rm -rf backend/.git
```

IMPORTANTE: Esto NO borra archivos de código. Solo elimina el historial git interno del backend. Como aun no tenia commits, no se pierden datos.

PASO 3: INICIALIZAR GIT EN LA RAIZ (EL MONOREPO)

Crea un unico repositorio git en la carpeta raiz. Este sera el unico .git que controle todo el proyecto.

```
git init
```

PASO 4: CREAR UN .GITIGNORE RAIZ

Es fundamental ignorar archivos que no deben versionarse: dependencias, builds, variables de entorno, bases de datos locales, etc.

```
cat > .gitignore << 'EOF'
# Dependencias
**/node_modules/
**/dist/
**/.env
**/.env.local
**/*.sqlite
**/data.sqlite

# Logs y builds
*.log
**/build/
**/coverage/

# IDE
.vscode/
.idea/

# Sistema operativo
.DS_Store
Thumbs.db
EOF
```

NOTA: El prefijo ****/** indica que aplica en cualquier subcarpeta, ya sea backend o frontend.

PASO 5: REVISAR QUE SE VA A COMMITTEAR

Antes de agregar archivos, siempre revisa el estado. Esto te permite detectar si algo se esta versionando por error.

```
git status
```

Deberias ver algo como:

```
Untracked files:
  .gitignore
  backend/
  frontend/
```

PASO 6: AGREGAR TODO Y COMMITEAR

Hay dos formas de hacerlo. La mas rapida es un solo commit. La mas prolija es por partes.

Opcion A: Todo en un solo commit

```
git add -A
git commit -m "init: monorepo PORTAL_DE_HERRAMIENTAS"

# O con mensaje detallado:
git commit -m "init: monorepo PORTAL_DE_HERRAMIENTAS" -m "- Backend: NestJS base" -m "- Frontend: React/Vite"
```

Opcion B: Por partes (mas prolijo)

```
# Primero el backend
git add backend/ .gitignore
git commit -m "feat: agregar backend NestJS"

# Luego el frontend
git add frontend/
git commit -m "feat: agregar frontend React/Vite"
```

PASO 7: VERIFICAR EL RESULTADO

Comproba que los commits quedaron bien y que la estructura es correcta.

```
git log --oneline
```

Deberia mostrar 1 o 2 commits, segun la opcion elegida.

```
ls -la
```

Deberias ver:

```
backend/  
frontend/  
.git/  
.gitignore
```

PASO 8: ABRIR EL REPO EN FORK

Para trabajar visualmente con el monorepo desde Fork:

1. En Fork: File -> Add Existing Repository
2. Selecciona la carpeta raiz (ej: PORTAL_DE_HERRAMIENTAS)
3. Listo — ahora tenes una sola ventana con todo el proyecto.

NOTA: A partir de ahora, todos los commits del backend y frontend aparecen en una sola linea de tiempo.

RESUMEN RAPIDO

1. `cd "ruta/al/proyecto"`
2. `rm -rf backend/.git`
3. `git init`
4. Crear `.gitignore` raiz
5. `git status` (verificar)
6. `git add -A && git commit -m "init: monorepo..."`
7. `git log --oneline` (confirmar)
8. Abrir en Fork

Comandos clave para no olvidar:

```
git add -A          # Agregar todos los cambios
git commit -m "..." # Commitear con mensaje
git status          # Ver estado antes de commitear
git log --oneline   # Ver historial resumido
```